# Peer Code Review and Replication Notes

Tammy Dang

## Code Review

### Code Review and Efficiency (10 points): Are there any parts of the code that seem inefficient or redundant? How could these sections be optimized for better performance or clarity?

For loading the HCRIS_1996 and HCRIS_2010 data, having them on two separate jupyter notebooks seemed a bit redundant. We can easily fix this by combining the two to be on the same notebook for easier loading and a faster process. In addition, the having many separate code chunks to run the code to answer one question was slightly inefficient, but could overall be resolved using the same technique. They also did not use a quarto markdown document, which made me have to switch the document into .qmd file. The data loading processing was repetitive and could be reformulated into a for-loop to load each dataset.

For the code itself, there is a redundant creating of intermediate datasets (df_plot, df_pen, df, df_clean) with similar transformations. This makes it memory intensive and harder to follow overall. In addition, removing duplicates earlier in the workflow would help with reptitive construction of the dataset

### Comparison of Results (10 points): How well do their results compare with your own? Are there any areas where the two analyses are particularly different?

We had similar results on most of the questions. The main difference was some of mine was log transformed and had different y-scales that made it look a bit different. The biggest difference definitely have to be the IV estimates, where Danzilo got completely different intercepts and results than my own.

### Replicability and Documentation (10 points): Are you able to clone your peer's repository and replicate their results?

Overall, thanks to their provided README file, I was able to easily clone and replicate most of their work on my own quarto markdown document. However, the one thing I had trouble with was loading the 2010 HCRIS data with their code. I had to retrace from my own code to be able to load the HCRIS 2010 year dataset. This caused some problems, especially with loading the graphs for question 5.

The one and only thing I had trouble with was loading the 2010 HCRIS data, as stated previously. The central problem was how their HCRIS_2010.ipynb code was structured. There are also no comments from each of the code files, which made it harder to follow and figure out as a reviewer. And since my work was in R while Danizlo's was in Python, this difference especially made it more difficult to follow along to the code's workflow.

In addition, their IV estimates when printing on a qmd went off the page, so having a code to adjust for the pdf size would be great for output readability.

### Final Thoughts (10 points): How might you update your own code based on your review of your peer's work?

Definitely have a clearer workflow for outputs. I really liked how Danzilo implemented the outputs within the folder. I had a pretty inefficient workflow for outputs that were outside of the actual git repository. Danzilo's was much more smoother in that regard, and I will replicate that for my future work.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import statsmodels.formula.api as smf
from pathlib import Path
import os
os.chdir('/Users/tammydang/Downloads/EMORY/ECON-470/Danzilo_G_HW_4')
```

```
raw_data = pd.read_csv('final_hcris.csv')
data_2008 = pd.read_csv('data/output/data-2008.csv')
data_2009 = pd.read_csv('data/output/data-2009.csv')
data_2010 = pd.read_csv('data/output/data-2010.csv')
data_2011 = pd.read_csv('data/output/data-2011.csv')
data_2012 = pd.read_csv('data/output/data-2012.csv')
data_2013 = pd.read_csv('data/output/data-2013.csv')
```

```
data_2014 = pd.read_csv('data/output/data-2014.csv')
data_2015 = pd.read_csv('data/output/data-2015.csv')
data_2016 = pd.read_csv('data/output/data-2016.csv')
data_2017 = pd.read_csv('data/output/data-2017.csv')
data_2018 = pd.read_csv('data/output/data-2018.csv')
data_2019 = pd.read_csv('data/output/data-2019.csv')
data = pd.concat([data_2009, data_2010, data_2011, data_2012, data_2013, data_2014, data_201
```

## Question 1

```
question_1 = (
    raw_data
    .groupby(["fyear", "provider_number"])
    .size()
    .reset_index(name="num_reports")
)
multiple_reports = question_1[question_1["num_reports"] > 1]
multiple_reports.head()
```

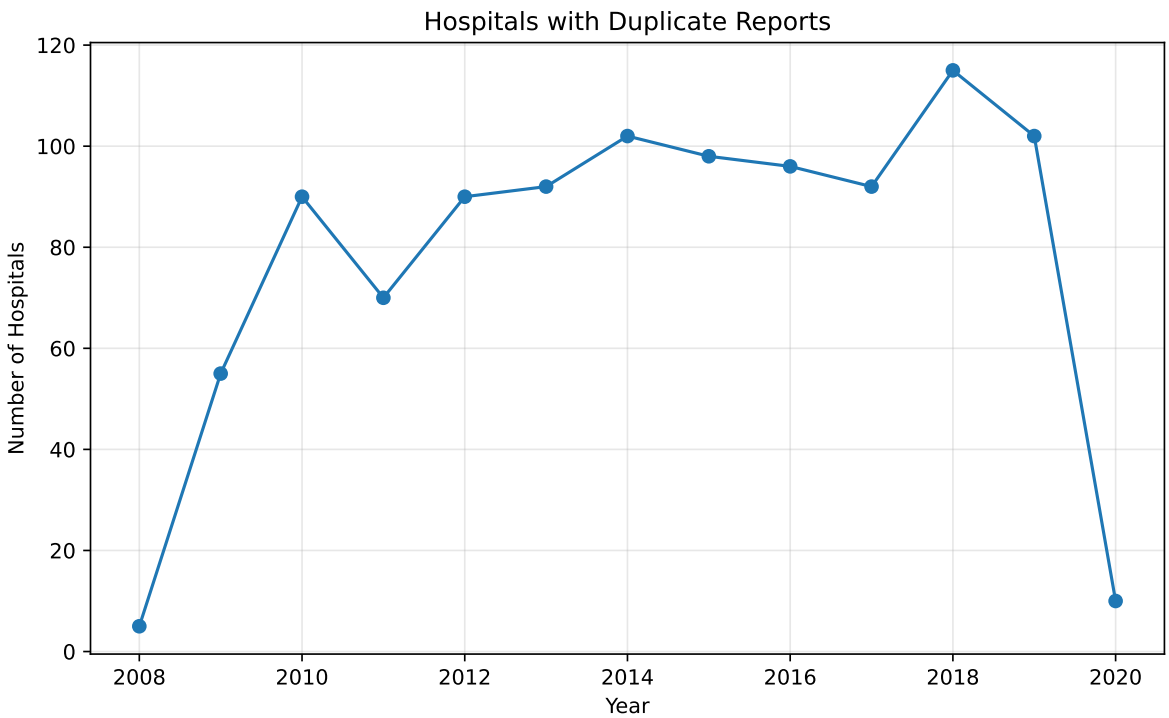
| | fyear | provider_number | num_reports |
|---|---|---|---|
| 106 | 2008 | 30061 | 2 |
| 118 | 2008 | 30093 | 2 |
| 182 | 2008 | 40151 | 2 |
| 259 | 2008 | 50158 | 2 |
| 271 | 2008 | 50230 | 2 |

```
hospitals_per_year = (
    multiple_reports
    .groupby("fyear")
    .size()
    .reset_index(name="num_hospitals")
)
hospitals_per_year
```

| | fyear | num_hospitals |
|---|---|---|
| 0 | 2008 | 42 |

| | fyear | num_hospitals |
|---|---|---|
| 1 | 2011 | 45 |
| 2 | 2012 | 90 |
| 3 | 2013 | 91 |
| 4 | 2014 | 102 |
| 5 | 2015 | 96 |
| 6 | 2016 | 95 |
| 7 | 2017 | 91 |
| 8 | 2018 | 101 |

```
prof_graph = pd.DataFrame({
    "year": [2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017, 2018, 2019, 2020],
    "num_hospitals": [5, 55, 90, 70, 90, 92, 102, 98, 96, 92, 115, 102, 10]
})

fig, ax = plt.subplots(figsize=(8, 5))

ax.plot(
    prof_graph["year"],
    prof_graph["num_hospitals"],
    marker="o"
)

ax.set(
    xlabel="Year",
    ylabel="Number of Hospitals",
    title="Hospitals with Duplicate Reports"
)

ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




## Question 2

```
data['provider_number'].nunique()
```

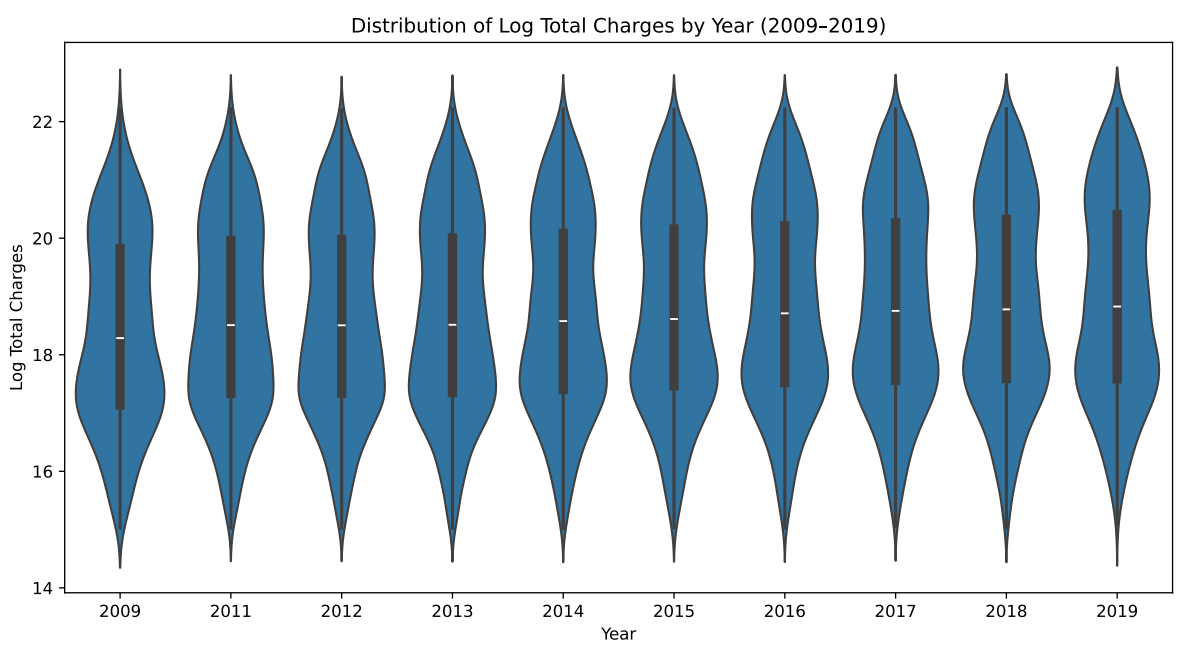
6782

## Question 3

```
df_plot = raw_data[(raw_data["fyear"] >= 2009) & (raw_data["fyear"] <= 2019)].copy()

df_plot["tot_charges"] = pd.to_numeric(df_plot["tot_charges"], errors="coerce")

df_plot = df_plot.dropna(subset=["tot_charges", "fyear"])
df_plot = df_plot[df_plot["tot_charges"] > 0]

lower = df_plot["tot_charges"].quantile(0.01)
```

```
upper = df_plot["tot_charges"].quantile(0.99)
df_plot = df_plot[
    (df_plot["tot_charges"] >= lower) &
    (df_plot["tot_charges"] <= upper)
].copy()

df_plot["log_tot_charges"] = np.log(df_plot["tot_charges"])
```

```
plt.figure(figsize=(12, 6))
sns.violinplot(
    data=df_plot,
    x="fyear",
    y="log_tot_charges"
)

plt.title("Distribution of Log Total Charges by Year (2009-2019)")
plt.xlabel("Year")
plt.ylabel("Log Total Charges")
plt.show()
```



## Question 4

```
raw_data["discount_factor"] = 1 - (raw_data["tot_discounts"] / raw_data["tot_charges"])

raw_data["price_num"] = (
    raw_data["ip_charges"] +
    raw_data["icu_charges"] +
    raw_data["ancillary_charges"]
) * raw_data["discount_factor"] - raw_data["tot_mcare_payment"]

raw_data["price_denom"] = raw_data["tot_discharges"] - raw_data["mcare_discharges"]

raw_data["price"] = raw_data["price_num"] / raw_data["price_denom"]
```

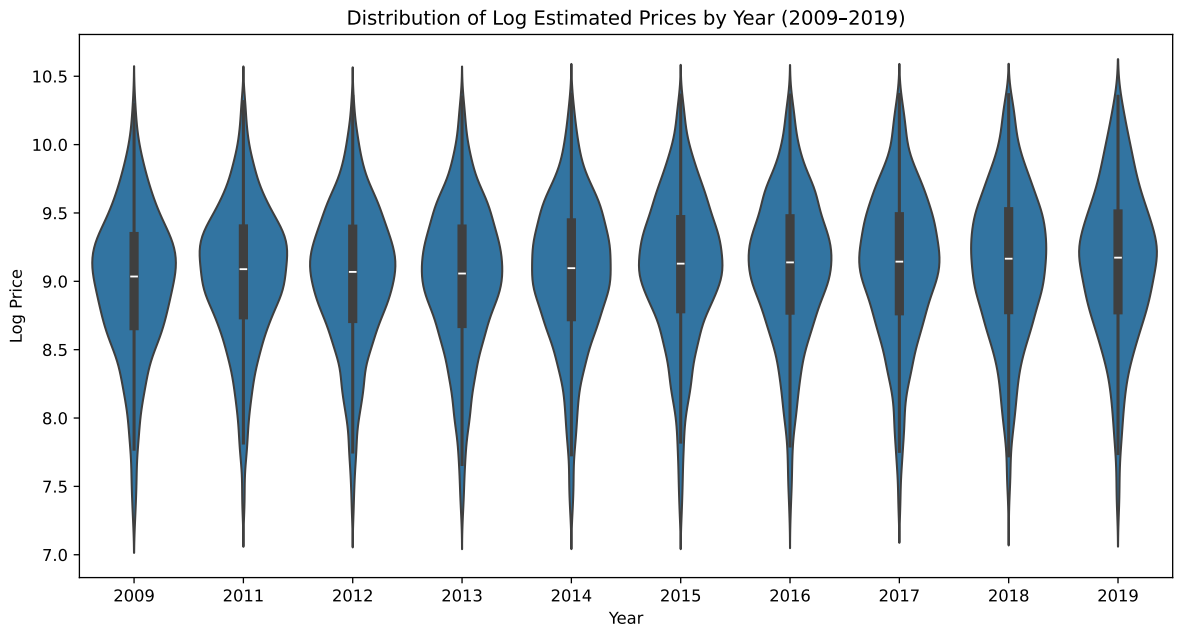
```
df_plot = raw_data.copy()

df_plot = df_plot[(df_plot["fyear"] >= 2009) & (df_plot["fyear"] <= 2019)]

df_plot = df_plot.replace([np.inf, -np.inf], np.nan)
df_plot = df_plot.dropna(subset=["price"])

df_plot = df_plot[df_plot["price"] > 0]
```

```
lower = df_plot["price"].quantile(0.01)
upper = df_plot["price"].quantile(0.99)

df_plot = df_plot[
    (df_plot["price"] >= lower) &
    (df_plot["price"] <= upper)
].copy()
```

```
df_plot["log_price"] = np.log(df_plot["price"])
```

```
plt.figure(figsize=(12, 6))
sns.violinplot(
    data=df_plot,
    x="fyear",
    y="log_price"
)
```

```
plt.title("Distribution of Log Estimated Prices by Year (2009-2019)")
plt.xlabel("Year")
plt.ylabel("Log Price")

plt.show()
```

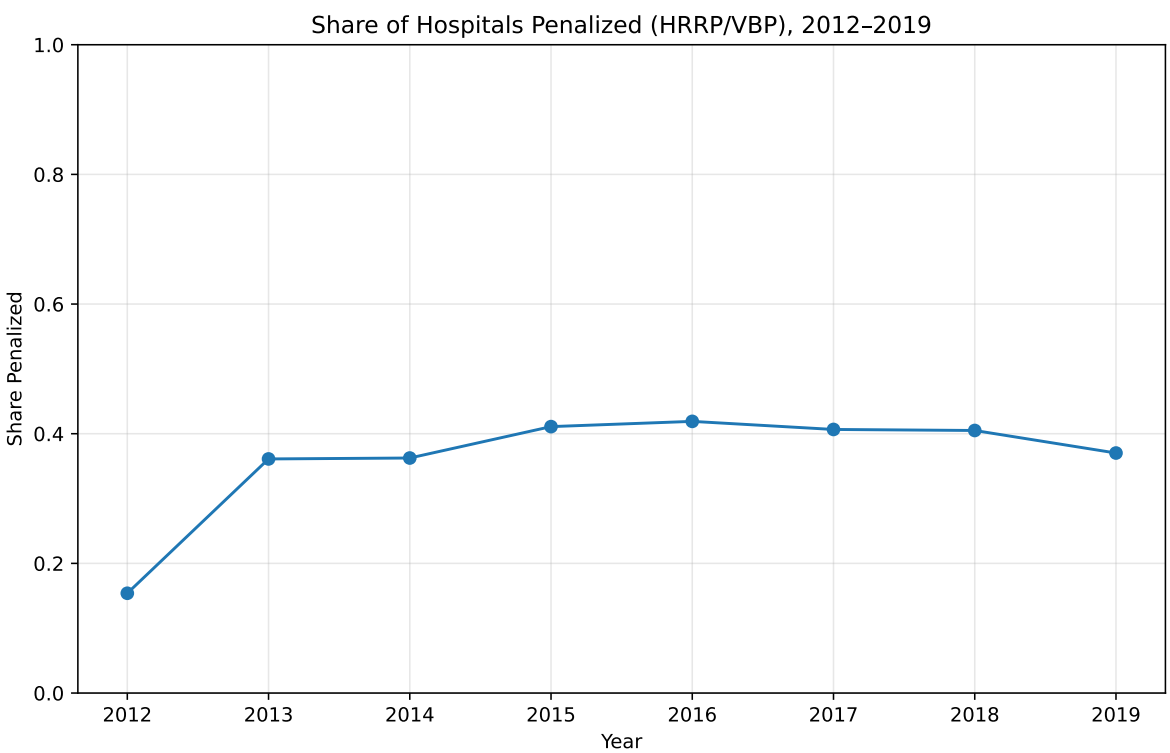



## Question 5

```
df_pen = raw_data.copy()

df_pen = df_pen[(df_pen["fyear"] >= 2012) & (df_pen["fyear"] <= 2019)]

df_pen["hrrp_payment"] = pd.to_numeric(df_pen["hrrp_payment"], errors="coerce")
df_pen["hvbp_payment"] = pd.to_numeric(df_pen["hvbp_payment"], errors="coerce")

df_pen["hrrp_payment"] = df_pen["hrrp_payment"].fillna(0)
df_pen["hvbp_payment"] = df_pen["hvbp_payment"].fillna(0)

df_pen["net_payment"] = df_pen["hrrp_payment"] + df_pen["hvbp_payment"]
```

```python
df_pen["penalized"] = (df_pen["net_payment"] > 0).astype(int)

share_pen = (
    df_pen.groupby("fyear")["penalized"]
    .mean()
    .reset_index()
)
```

```python
plt.figure(figsize=(10, 6))

plt.plot(share_pen["fyear"], share_pen["penalized"], marker="o")

plt.title("Share of Hospitals Penalized (HRRP/VBP), 2012-2019")
plt.xlabel("Year")
plt.ylabel("Share Penalized")
plt.ylim(0, 1)
plt.grid(True, alpha=0.3)

plt.show()
```



## Question 6

```
df = data.copy()

df["hrrp_payment"] = pd.to_numeric(df["hrrp_payment"], errors="coerce").abs().fillna(0)
df["hvbp_payment"] = pd.to_numeric(df["hvbp_payment"], errors="coerce").fillna(0)
df["net_penalty"]  = df["hvbp_payment"] - df["hrrp_payment"]

num_vars = [
    "tot_discounts", "tot_charges", "ip_charges", "icu_charges",
    "ancillary_charges", "tot_mcare_payment", "tot_discharges",
    "mcare_discharges", "mcaid_discharges", "beds"
]
for v in num_vars:
    df[v] = pd.to_numeric(df[v], errors="coerce")

df["discount_factor"] = 1 - df["tot_discounts"] / df["tot_charges"]
df["price_num"] = (
    (df["ip_charges"] + df["icu_charges"] + df["ancillary_charges"])
    * df["discount_factor"]
    - df["tot_mcare_payment"]
)
df["price_denom"] = df["tot_discharges"] - df["mcare_discharges"]
df["price"] = df["price_num"] / df["price_denom"]
df = df.replace([np.inf, -np.inf], np.nan)
df.loc[df["price"] <= 0, "price"] = np.nan

price_2011 = (
    df.loc[df["year"] == 2011, ["provider_number", "price"]]
    .rename(columns={"price": "price_2011"})
)
price_2014 = (
    df.loc[df["year"] == 2014, ["provider_number", "price"]]
    .rename(columns={"price": "price_2014"})
)
pen_2012 = (
    df.loc[df["year"] == 2012, ["provider_number", "net_penalty"]]
    .rename(columns={"net_penalty": "net_penalty_2012"})
)

pre_period = df.loc[df["year"].between(2009, 2011)].copy()
```

```python
pre_means = (
    pre_period.groupby("provider_number", as_index=False)
    .agg(
        mean_beds_pre=("beds", "mean"),
        mean_mcaid_discharges_pre=("mcaid_discharges", "mean"),
        mean_mcare_discharges_pre=("mcare_discharges", "mean"),
    )
)

analysis = (
    price_2011
    .merge(price_2014, on="provider_number", how="inner")
    .merge(pen_2012,   on="provider_number", how="inner")
    .merge(pre_means,  on="provider_number", how="inner")
)
analysis["price_change"] = analysis["price_2014"] - analysis["price_2011"]

print("N in analysis:", len(analysis))
```

```
N in analysis: 5557
```

```python
print("price_2011:", len(price_2011))
print("price_2014:", len(price_2014))
print("pen_2012:", len(pen_2012))
print("pre_means:", len(pre_means))

print("dup in price_2011:", price_2011["provider_number"].duplicated().sum())
print("dup in price_2014:", price_2014["provider_number"].duplicated().sum())
print("dup in pen_2012:", pen_2012["provider_number"].duplicated().sum())
print("dup in pre_means:", pre_means["provider_number"].duplicated().sum())
```

```
price_2011: 5815
price_2014: 6064
pen_2012: 6140
pre_means: 6245
dup in price_2011: 0
dup in price_2014: 0
dup in pen_2012: 0
dup in pre_means: 0
```

```python
df_clean = (
    df.dropna(subset=["price"])
      .sort_values("provider_number")
      .drop_duplicates(subset=["provider_number", "year"], keep="first")
)

price_2011 = (
    df_clean.loc[df_clean["year"] == 2011, ["provider_number", "price"]]
    .rename(columns={"price": "price_2011"})
)
price_2014 = (
    df_clean.loc[df_clean["year"] == 2014, ["provider_number", "price"]]
    .rename(columns={"price": "price_2014"})
)
pen_2012 = (
    df_clean.loc[df_clean["year"] == 2012, ["provider_number", "net_penalty"]]
    .rename(columns={"net_penalty": "net_penalty_2012"})
)

pre_period = df_clean.loc[df_clean["year"].between(2009, 2011)].copy()
pre_means = (
    pre_period.groupby("provider_number", as_index=False)
    .agg(
        mean_beds_pre=("beds", "mean"),
        mean_mcaid_discharges_pre=("mcaid_discharges", "mean"),
        mean_mcare_discharges_pre=("mcare_discharges", "mean"),
    )
)

analysis = (
    price_2011
    .merge(price_2014, on="provider_number", how="inner")
    .merge(pen_2012,   on="provider_number", how="inner")
    .merge(pre_means,  on="provider_number", how="inner")
)
analysis["price_change"] = analysis["price_2014"] - analysis["price_2011"]

print("N in analysis:", len(analysis))
print("price_2011:", len(price_2011))
```

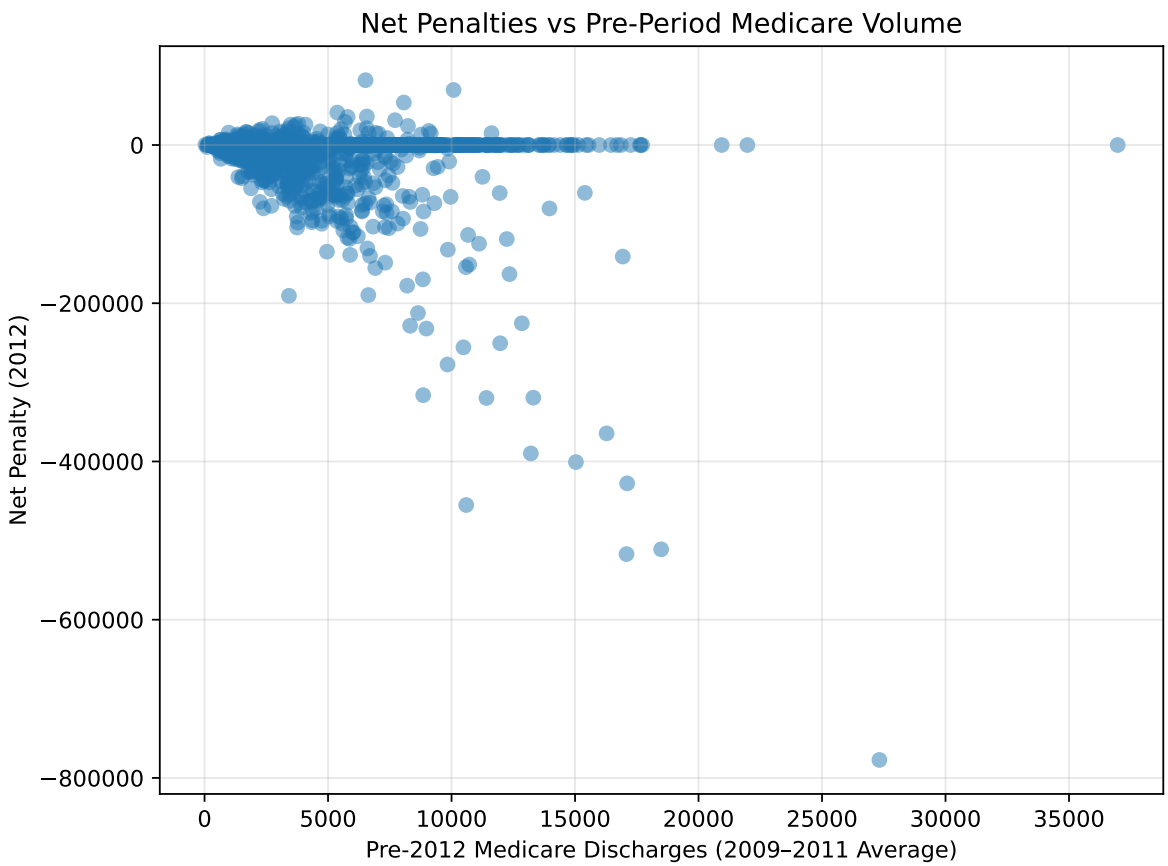
```
N in analysis: 2376
price_2011: 2621
```

```python
from statsmodels.iolib.summary2 import summary_col

analysis_ols = analysis.dropna(subset=[
    "price_change",
    "net_penalty_2012",
    "mean_beds_pre",
    "mean_mcaid_discharges_pre"
]).copy()

ols1 = smf.ols("price_change ~ net_penalty_2012",
               data=analysis_ols).fit(cov_type="HC1")
ols2 = smf.ols("price_change ~ net_penalty_2012 + mean_beds_pre",
               data=analysis_ols).fit(cov_type="HC1")
ols3 = smf.ols("price_change ~ net_penalty_2012 + mean_beds_pre + mean_mcaid_discharges_pre"
               data=analysis_ols).fit(cov_type="HC1")

print(summary_col([ols1, ols2, ols3], stars=True,
                  model_names=["(1)", "(2)", "(3)"],
                  info_dict={"N": lambda x: f"{int(x.nobs)}",
                             "R2": lambda x: f"{x.rsquared:.3f}"}))
```

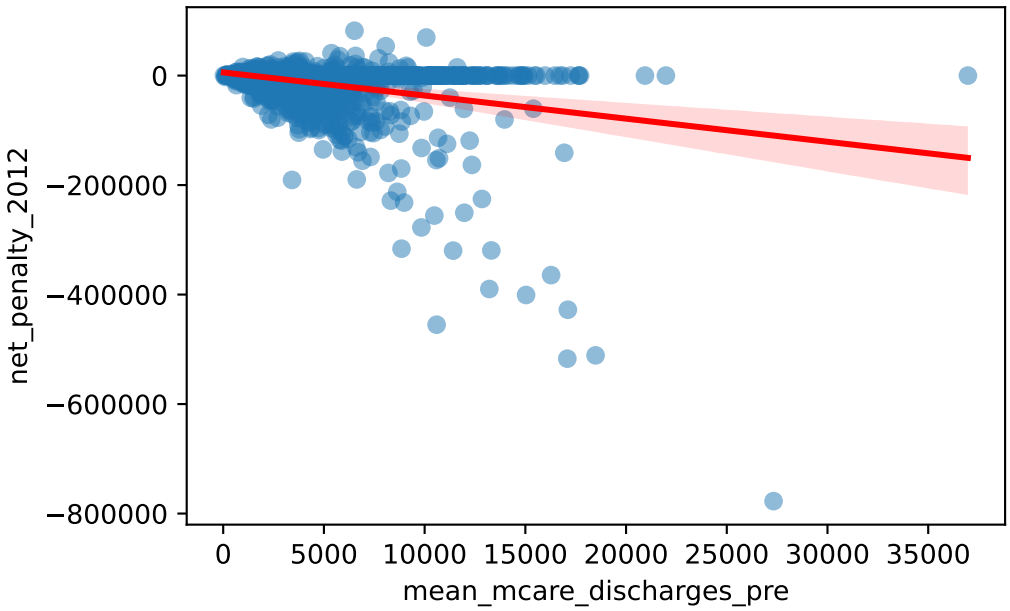
```
==========================================================
                            (1)        (2)        (3)
----------------------------------------------------------
Intercept                765.3082*  -199.0538  -210.6491
                         (405.5461) (389.3402) (404.0098)
net_penalty_2012         -0.0027    0.0045     0.0059
                         (0.0028)   (0.0079)   (0.0099)
mean_beds_pre                       4.6443     6.5053
                                    (3.6381)   (6.2611)
mean_mcaid_discharges_pre                      -0.2384
                                               (0.3503)
R-squared                0.0000     0.0021     0.0025
R-squared Adj.           -0.0004    0.0012     0.0012
N                        2366       2366       2366
R2                       0.000      0.002      0.002
==========================================================
Standard errors in parentheses.
* p<.1, ** p<.05, ***p<.01
```

## Question 7

```
plt.figure(figsize=(8, 6))

plt.scatter(
    analysis["mean_mcare_discharges_pre"],
    analysis["net_penalty_2012"],
    alpha=0.5
)

plt.xlabel("Pre-2012 Medicare Discharges (2009-2011 Average)")
plt.ylabel("Net Penalty (2012)")
plt.title("Net Penalties vs Pre-Period Medicare Volume")

plt.grid(True, alpha=0.3)

plt.show()
```




```
sns.regplot(
    data=analysis,
    x="mean_mcare_discharges_pre",
    y="net_penalty_2012",
    scatter_kws={"alpha": 0.5},
    line_kws={"color": "red"}
)
```




## Question 8

```
fs1 = smf.ols("net_penalty_2012 ~ mean_mcare_discharges_pre", data=analysis).fit(cov_type="HC
fs2 = smf.ols("net_penalty_2012 ~ mean_mcare_discharges_pre + mean_beds_pre", data=analysis)
fs3 = smf.ols("net_penalty_2012 ~ mean_mcare_discharges_pre + mean_beds_pre + mean_mcaid_disc

rf1 = smf.ols("price_change ~ mean_mcare_discharges_pre", data=analysis).fit(cov_type="HC1")
rf2 = smf.ols("price_change ~ mean_mcare_discharges_pre + mean_beds_pre", data=analysis).fit
rf3 = smf.ols("price_change ~ mean_mcare_discharges_pre + mean_beds_pre + mean_mcaid_dischar

print(summary_col([fs1, fs2, fs3, rf1, rf2, rf3], stars=True,
                  model_names=["FS(1)","FS(2)","FS(3)","RF(1)","RF(2)","RF(3)"],
                  info_dict={"N": lambda x: f"{int(x.nobs)}",
                             "R2": lambda x: f"{x.rsquared:.3f}"}))
```

```
=================================================================================================
                              FS(1)       FS(2)       FS(3)       RF(1)      RF(2)      RF(3)
-------------------------------------------------------------------------------------------------
Intercept                  5757.3937** 6395.7690** 6193.2488** -1049.6091  -1105.4158 -250.51
                           (2812.7280) (3168.1571) (3067.0605) (1039.0023) (996.8604) (515.50
```

```
mean_mcare_discharges_pre -4.2213***  -3.0951***  -2.4529***  0.4017      0.3032      0.1666
                          (0.9248)    (0.8676)    (0.8825)    (0.2962)    (0.4698)    (0.4129)
mean_beds_pre                         -21.3724    -51.9072**              1.8684      3.2904*
                                      (18.8010)   (25.9560)               (3.9116)    (1.7830)
mean_mcaid_discharges_pre                         2.8438***                           -0.1846
                                                  (0.9529)                            (0.2370)
R-squared                 0.1137      0.1161      0.1283      0.0024      0.0024      0.0025
R-squared Adj.            0.1133      0.1154      0.1272      0.0020      0.0016      0.0012
N                         2376        2376        2366        2376        2376        2366
R2                        0.114       0.116       0.128       0.002       0.002       0.003
==========================================================================================
Standard errors in parentheses.
* p<.1, ** p<.05, ***p<.01
```

## Question 9

```python
from linearmodels.iv import IV2SLS

analysis_iv = analysis.dropna(subset=[
    "price_change", "net_penalty_2012",
    "mean_mcare_discharges_pre", "mean_beds_pre", "mean_mcaid_discharges_pre"
]).copy()

iv1 = IV2SLS.from_formula(
    "price_change ~ 1 + [net_penalty_2012 ~ mean_mcare_discharges_pre]",
    data=analysis_iv).fit(cov_type="robust")

iv2 = IV2SLS.from_formula(
    "price_change ~ 1 + mean_beds_pre + [net_penalty_2012 ~ mean_mcare_discharges_pre]",
    data=analysis_iv).fit(cov_type="robust")

iv3 = IV2SLS.from_formula(
    "price_change ~ 1 + mean_beds_pre + mean_mcaid_discharges_pre + [net_penalty_2012 ~ mean_
    data=analysis_iv).fit(cov_type="robust")

print(iv1.summary)
print(iv2.summary)
print(iv3.summary)
```

```
                    IV-2SLS Estimation Summary
```

```
==============================================================================
Dep. Variable:           price_change   R-squared:                     -0.0180
Estimator:                    IV-2SLS   Adj. R-squared:                -0.0185
No. Observations:                2366   F-statistic:                    1.0101
Date:                Thu, Apr 23 2026   P-value (F-stat)                0.3149
Time:                        15:59:11   Distribution:                  chi2(1)
Cov. Estimator:                robust

                              Parameter Estimates
==================================================================================
                  Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
----------------------------------------------------------------------------------
Intercept            153.61     294.22     0.5221     0.6016     -423.05      730.27
net_penalty_2012    -0.0657     0.0653    -1.0050     0.3149     -0.1938      0.0624
==================================================================================

Endogenous: net_penalty_2012
Instruments: mean_mcare_discharges_pre
Robust Covariance (Heteroskedastic)
Debiased: False
                          IV-2SLS Estimation Summary
==============================================================================
Dep. Variable:           price_change   R-squared:                     -0.0190
Estimator:                    IV-2SLS   Adj. R-squared:                -0.0198
No. Observations:                2366   F-statistic:                    4.3325
Date:                Thu, Apr 23 2026   P-value (F-stat)                0.1146
Time:                        15:59:11   Distribution:                  chi2(2)
Cov. Estimator:                robust

                              Parameter Estimates
==================================================================================
                  Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
----------------------------------------------------------------------------------
Intercept            166.86     549.84     0.3035     0.7615     -910.80      1244.5
mean_beds_pre       -0.1308     7.2518    -0.0180     0.9856     -14.344      14.083
net_penalty_2012    -0.0673     0.1514    -0.4446     0.6566     -0.3640      0.2294
==================================================================================

Endogenous: net_penalty_2012
Instruments: mean_mcare_discharges_pre
Robust Covariance (Heteroskedastic)
Debiased: False
                          IV-2SLS Estimation Summary
```

```
==============================================================================
Dep. Variable:           price_change   R-squared:                     -0.0193
Estimator:                    IV-2SLS   Adj. R-squared:                -0.0206
No. Observations:                2366   F-statistic:                    4.3200
Date:                Thu, Apr 23 2026   P-value (F-stat)                0.2289
Time:                        15:59:11   Distribution:                  chi2(3)
Cov. Estimator:                robust

                                   Parameter Estimates
=============================================================================================
                           Parameter  Std. Err.     T-stat    P-value    Lower CI    Upper CI
---------------------------------------------------------------------------------------------
Intercept                     170.15     644.38     0.2640     0.7917     -1092.8      1433.1
mean_beds_pre                -0.2352     10.511    -0.0224     0.9821     -20.836      20.365
mean_mcaid_discharges_pre     0.0086     0.2826     0.0304     0.9758     -0.5453      0.5625
net_penalty_2012             -0.0679     0.1700    -0.3996     0.6894     -0.4011      0.2652
=============================================================================================

Endogenous: net_penalty_2012
Instruments: mean_mcare_discharges_pre
Robust Covariance (Heteroskedastic)
Debiased: False
```

```python
from linearmodels.iv import IV2SLS

analysis_iv = analysis.dropna(subset=[
    "price_change", "net_penalty_2012",
    "mean_mcare_discharges_pre", "mean_beds_pre", "mean_mcaid_discharges_pre"
]).copy()

iv1 = IV2SLS.from_formula(
    "price_change ~ 1 + [net_penalty_2012 ~ mean_mcare_discharges_pre]",
    data=analysis_iv).fit(cov_type="robust")

iv2 = IV2SLS.from_formula(
    "price_change ~ 1 + mean_beds_pre + [net_penalty_2012 ~ mean_mcare_discharges_pre]",
    data=analysis_iv).fit(cov_type="robust")

iv3 = IV2SLS.from_formula(
    "price_change ~ 1 + mean_beds_pre + mean_mcaid_discharges_pre + [net_penalty_2012 ~ mean_
    data=analysis_iv).fit(cov_type="robust")
```

```python
# Print clean table
iv_table = pd.DataFrame({
    "(1)": [iv1.params.get("Intercept", np.nan),
            iv1.std_errors.get("Intercept", np.nan),
            iv1.params.get("net_penalty_2012", np.nan),
            iv1.std_errors.get("net_penalty_2012", np.nan),
            np.nan, np.nan, np.nan, np.nan,
            int(iv1.nobs), iv1.rsquared],
    "(2)": [iv2.params.get("Intercept", np.nan),
            iv2.std_errors.get("Intercept", np.nan),
            iv2.params.get("net_penalty_2012", np.nan),
            iv2.std_errors.get("net_penalty_2012", np.nan),
            iv2.params.get("mean_beds_pre", np.nan),
            iv2.std_errors.get("mean_beds_pre", np.nan),
            np.nan, np.nan,
            int(iv2.nobs), iv2.rsquared],
    "(3)": [iv3.params.get("Intercept", np.nan),
            iv3.std_errors.get("Intercept", np.nan),
            iv3.params.get("net_penalty_2012", np.nan),
            iv3.std_errors.get("net_penalty_2012", np.nan),
            iv3.params.get("mean_beds_pre", np.nan),
            iv3.std_errors.get("mean_beds_pre", np.nan),
            iv3.params.get("mean_mcaid_discharges_pre", np.nan),
            iv3.std_errors.get("mean_mcaid_discharges_pre", np.nan),
            int(iv3.nobs), iv3.rsquared],
}, index=["Intercept", "Intercept SE",
          "Net Penalty (IV)", "Net Penalty SE",
          "Avg Beds (Pre-2012)", "Beds SE",
          "Avg Medicaid Discharges (Pre-2012)", "Medicaid SE",
          "Num.Obs.", "R2"])

print(iv_table.round(3))
```

```
                                        (1)       (2)       (3)
Intercept                           153.611   166.864   170.147
Intercept SE                        294.218   549.837   644.384
Net Penalty (IV)                     -0.066    -0.067    -0.068
Net Penalty SE                        0.065     0.151     0.170
Avg Beds (Pre-2012)                     NaN    -0.131    -0.235
Beds SE                                 NaN     7.252    10.511
Avg Medicaid Discharges (Pre-2012)      NaN       NaN     0.009
Medicaid SE                             NaN       NaN     0.283
```

```
Num.Obs.                          2366.000  2366.000  2366.000
R2                                  -0.018    -0.019    -0.019
```

## Question 10

The IV estimate identifies a local ATE, reflecting the effect of penalties on prices for hospitals whose penalty status is influenced affected by their pre-2012 Medicare discharges. This effect applies mostly to hospitals with higher Medicare exposure and can be different from overall ATE, reflecing the average effect across all hospitals.